Bioinformatics, Original Paper, Sequence analysis — draft

dinara-align: hardware-aware algorithms for exact pairwise alignment

Konstantinos Kyriakidis[1,*] and Benedict Paten[1,*]
[1]UC Santa Cruz Genomics Institute, Santa Cruz, CA, USA
[*]To whom correspondence should be addressed: kkyriaki@ucsc.edu, bpaten@ucsc.edu

Motivation: Exact pairwise alignment of long sequencing reads remains computationally demanding. Band doubling over bit-parallel blocks, the wavefront algorithm and A* with seed heuristics have greatly reduced the number of cells an exact aligner must compute. A large part of the remaining running time depends on how the computation maps onto the processor: loop-carried dependency chains, memory-bound table lookups, and bounds and thresholds that are poorly matched to the input or the hardware.
Results: We present dinara-align, an exact pairwise aligner built on algorithmic changes that address these bottlenecks: a direct construction of the neighbours of exact seed matches that provably preserves local pruning, diagonal transition vectorised across diagonals, band bounds accepted only when independent estimates agree, and per-pair optimality certificates for banded inter-sequence batches, together with a bit-parallel sweep arranged for instruction latency. On an Intel Skylake-X processor, dinara-align had the lowest mean running time among the exact aligners tested on all 22 datasets of the A*PA2 and QuickEd benchmarks, aligning ultra-long nanopore reads in 118 ms on average, compared with 283 ms for A*PA2-full, and was 6.7 times faster than QuickEd in the geometric mean. It was 1.7 to 3.8 times faster than WFA and BiWFA at gap-affine costs and computed global scores for 500,000 short-read pairs on one core 4.6 times faster than WFA2-lib.
Availability and implementation: dinara-align is free software under the Mozilla Public License 2.0, available at `https://github.com/kokyriakidis/dinara-align` with C, C++, Python and command-line interfaces.
Contact: kkyriaki@ucsc.edu, bpaten@ucsc.edu
Supplementary information: Supplementary data are available at *Bioinformatics* online.

1 Introduction

Pairwise alignment is a building block of read mapping, genome assembly and variant calling. It determines the minimum-cost series of substitutions, insertions and deletions that transforms one sequence into another, and the classical dynamic programming algorithm computes it over an $n \times m$ matrix (Gotoh, 1982). That is practical for short reads but not for current long reads: nanopore reads span tens to hundreds of kilobases, often with 5 to 15% errors, and one pair of 500 kbp reads fills a matrix of 2.5×10^{11} cells.

Heuristic aligners compute only part of the matrix, a band, a window or the cells before a drop in score, and lose the guarantee of optimality. Exact aligners instead skip only those cells that provably cannot lie on an optimal path, so that their running time depends on the distance between the sequences rather than on the product of their lengths.

Three ideas underlie current exact aligners. *Band doubling* computes only the diagonals that a path within a given cost bound can reach, and doubles the bound until the optimal cost lies within it (Ukkonen, 1985). *Bit-parallelism* encodes a column of 64 cells in one machine word (Myers, 1999), and Edlib combines it with band doubling (Šošić and Šikić, 2017). *Diagonal transition* extends the furthest-reaching paths one cost at a time, following runs of matches at no cost (Ukkonen, 1985; Myers, 1986); WFA generalised it to gap-affine costs (Marco-Sola et al., 2021) and BiWFA reduced its memory to linear in the distance (Marco-Sola et al., 2023). A*PA introduced A* search with a gap-chaining seed heuristic (Groot Koerkamp and Ivanov, 2024), and A*PA2 combined this heuristic with band doubling over SIMD blocks of bit-parallel words, aligning ultra-long reads up to 19 times faster than Edlib and BiWFA (Groot Koerkamp, 2024). QuickEd first bounds the distance with windowed and banded heuristics and then aligns within the bound (Doblas et al., 2025). A second line of work accelerates the full dynamic program on vector hardware, using anti-diagonal (Wozniak, 1997), striped (Farrar, 2007; Zhao et al., 2013) and prefix-scan (Daily, 2016) layouts, difference recurrences (Suzuki and Kasahara, 2018; Li, 2018), adaptive blocks (Liu and Steinegger, 2023) and inter-sequence parallelism (Rognes, 2011; Rahn et al., 2018); these methods compute the full matrix or a fixed band, so their running time grows with its size rather than with the distance.

In the exact methods, a large part of the remaining running time is determined by how the computation maps onto the processor. The seed heuristic is built by scalar lookups in tables larger than the caches. Diagonal transition extends one diagonal at a time. Band doubling spends whole rounds when its bound is far from the distance. Batches of short pairs are vectorised across pairs, but each pair's full matrix is computed. And the bit-parallel sweep carries a dependency from each column to the next, so its speed is set by the latency of that chain rather than by its number of operations. The choice among these algorithms rests on thresholds measured on earlier versions of the code, and the boundaries that a band keeps for its traceback take memory that grows with the distance. Here we present dinara-align, an exact aligner whose algorithms address these bottlenecks. Our contributions are:

1. **Direct neighbour construction for the seed heuristic.** The inexact neighbours of an exact seed match are inserted without search, and we prove

that this preserves the outcome of local pruning (Lemma 1).

2. **Gathered diagonal transition.** Diagonal transition and the gap-affine wavefront extend eight diagonals per step using gather instructions.

3. **Band bounds from agreeing estimates.** A band-doubling bound is extrapolated from partial progress only when two independent estimates agree.

4. **Certified bands for inter-sequence batches.** Each short pair is aligned in one vector lane within a narrow band whose optimality is certified per lane by a lower bound on the cost of any path that leaves it (Proposition 1), adapting the speculate-and-verify approach of the SeedEx accelerator (Fujiki et al., 2020) to software batches.

5. **A latency-aware bit-parallel sweep.** On AVX-512, two staggered groups of words are interleaved, extending the instruction-level parallelism of A*PA2, and two Boolean identities shorten the loop-carried chain of Myers' step; the two are worth 4.5 to 5% and 1 to 2% on the ultra-long reads, and 11.5% and 4% on 100 kbp pairs at 20% errors.

6. **A traceback within a memory limit that keeps the tie rule.** When the tile boundaries a band keeps for its traceback would exceed the memory limit, only some are kept, and the stretches between them are computed again from the recorded state of the band, so the alignment is the same byte for byte.

7. **Thresholds measured against the code.** Every threshold that selects an algorithm or bounds a search was measured again on the final code, which exposed thresholds that had cost up to a third of the running time.

Section 2 reviews the algorithms dinara-align builds on, Section 3 describes our changes, Section 4 compares it with the exact aligners of the A*PA2 and QuickEd benchmarks, and Section 5 discusses the results and their limitations.

2 Background

2.1 Alignment and costs

Let $A = a_1 \dots a_n$ be the reference and $B = b_1 \dots b_m$ the query, and let cell (i, j) lie on diagonal $j - i$. Under gap-affine costs, a mismatch costs x and a gap of k letters costs $g(k) = o + ke$; unit costs correspond to $x = e = 1$ and $o = 0$, and two-piece costs take the minimum of two such functions (Li, 2018). A substitution matrix instead assigns each pair of letters a score to be maximised. We consider global, ends-free, extension and local alignment. An alignment is *exact* if it is optimal under the requested costs, mode, band and cost limit; among optimal alignments, a fixed rule places indels leftmost or rightmost, so that, under costs, the result does not depend on which algorithm computed it (Section 3.1). At unit costs, the distance $D(i, j)$ between the prefixes $a_1 \dots a_i$ and $b_1 \dots b_j$ satisfies

$$D(i,j) = \min\big(D(i-1,j)+1,\ D(i,j-1)+1,\ D(i-1,j-1)+[a_i \neq b_j]\big), \tag{1}$$

with $D(i, 0) = i$ and $D(0, j) = j$, and the distance is $D(n, m)$; gap-affine costs need three such layers (Gotoh, 1982).

2.2 Band doubling and bit-parallel computation

An alignment of cost at most t visits only diagonals d with $|d| + |d - \delta| \leq t$, where $\delta = m - n$ is the diagonal on which it ends. Band doubling computes only those cells, for $t = t_0, 2t_0, \dots$, until $D(n, m) \leq t$, in $O(ns)$ time for distance s (Ukkonen, 1985). Myers (Myers, 1999) encodes the vertical differences of 64 consecutive cells of a column, each -1, 0 or $+1$, in two machine words P_v and M_v, and advances them by one column with about twenty bitwise operations. Edlib combines this encoding with band doubling (Šošić and Šikić, 2017). A*PA2 computes the band in tiles of 256 columns, several words at once in vector registers, and drops the rows of a tile whose score already rules them out (Groot Koerkamp, 2024).

2.3 Diagonal transition

Diagonal transition stores, for each cost s and diagonal d, the furthest cell on d that an alignment of cost s reaches, and extends it along matches at no cost; the distance is the first s whose wavefront reaches (n, m), after $O(n + s^2)$ steps (Ukkonen, 1985; Myers, 1986). WFA generalises it to gap-affine costs with three wavefronts per cost (Marco-Sola et al., 2021), and BiWFA runs it from both ends until the wavefronts meet, which keeps its memory linear in s (Marco-Sola et al., 2023).

2.4 The seed heuristic

A*PA divides the reference into disjoint seeds of k bases and finds their matches in the query, exactly or with one edit (Groot Koerkamp and Ivanov, 2024). Its gap-chaining seed heuristic bounds the cost from a cell to the end of the alignment by the seeds ahead that no chain of matches from that cell covers, counting the gaps between chained matches, and never overestimates. Local pruning discards a match ν unless $c(\nu) + h(e(\nu)) < \tau$, where $c(\nu)$ is its cost, $e(\nu)$ its end cell, $h(u)$ the least cost of a path from u across the following seeds, and τ a threshold that depends only on the seeds. A*PA2 uses the heuristic to drop the rows of each band whose score plus the heuristic exceeds the bound (Groot Koerkamp, 2024).

3 Methods

3.1 Overview

Each problem is assigned to an algorithm (Table 1; Fig. 1). For unit costs we follow A*PA2 (Groot Koerkamp, 2024): band doubling over tiles of 256 columns of bit-parallel words, pruned by the gap-chaining seed heuristic (Groot Koerkamp and Ivanov, 2024), with a diagonal-transition traceback inside each tile; near-identical pairs are aligned by diagonal transition alone. Gap-affine costs use the bidirectional wavefront of WFA and BiWFA (Marco-Sola et al., 2021, 2023). A short diagonal-transition search estimates the distance, and the estimate, the length and the cost model select the algorithm. The thresholds are fixed at compile time for each instruction set (Section 3.9).

Under costs, every engine selects the same alignment among equally good ones. All follow the backtrace order of WFA2-lib: from the end of the alignment, an edit is taken into a cell whenever one is optimal, a substitution before a deletion before an insertion, and a gap is extended before it is opened. The tile traceback applies this order cell by cell (Section 3.7), the batches and the difference sweep store per-cell flags under it, and the wavefronts take the furthest-reaching source, which makes the same choice. Leftmost placement of indels is obtained by running this rightmost rule on both sequences reversed and reversing the result. Two exceptions remain: a gap-affine pair whose wavefronts would exceed the memory limit is split where an optimal path crosses and follows the rule within each piece, and alignments under a substitution table follow the traceback order of Gotoh's recurrence.

3.2 A shorter critical path for Myers' recurrence

As in Edlib and A*PA2 (Šošić and Šikić, 2017; Groot Koerkamp, 2024), the query is packed 64 bases per word, and Myers' recurrence (Myers, 1999) advances a word by one column. With match mask E, vertical difference masks P_v, M_v and the incoming negative horizontal difference folded into $E' = E \vee M_h^{\text{in}}$, A*PA2 computes

$$\begin{aligned} c &= (E' \wedge P_v) + P_v, \\ X_h &= (c \oplus P_v) \vee E', \\ P_h &= M_v \vee \neg(X_h \vee P_v), \\ M_h &= P_v \wedge X_h. \end{aligned} \tag{2}$$

The horizontal masks P_h and M_h determine the vertical masks of the next column, so the path from c to them lies on the loop-carried chain. Since $(c \oplus P_v) \vee P_v = c \vee P_v$ and $P_v \wedge (c \oplus P_v) = P_v \wedge \neg c$, the same masks are obtained as

$$\begin{aligned} P_h &= M_v \vee \neg\big(c \vee (E' \vee P_v)\big), \\ M_h &= (P_v \wedge \neg c) \vee (P_v \wedge E'), \end{aligned} \tag{3}$$

where $E' \vee P_v$ and $P_v \wedge E'$ do not depend on c and are computed off the critical path. This removes two operations from the loop-carried chain and, for a single word, reduces the cost from 14 to 8 cycles per column on the Skylake-X. Myers (Myers, 1999), Edlib and A*PA2 use the original form, and we are not aware of a previous description of the regrouped one.

Words occupy the lanes of a vector, staggered by one column, so that the horizontal difference leaving one word enters the next through a lane rotation (Wozniak, 1997; Groot Koerkamp, 2024). Every column then waits for the rotation of the previous one, and A*PA2 processes two four-lane vectors together to overlap these waits. On AVX-512, where a vector holds eight words, dinara-align interleaves two eight-word groups, the lower lagging ten columns behind the upper, so that the instructions of each group fill the latency of the other; with narrower vectors a single group is used. Bands of two or three words, in which every operation lies on the critical path, are computed in general-purpose registers, whose operations have lower latency. On AVX-512 the sweep requires about 2.4 cycles per word and column.

3.3 Band bounds from agreeing estimates

Band doubling returns the exact distance for any initial bound, but its running time depends on how close that bound is to the distance. A partial computation that has reached cost s at anti-diagonal r suggests the extrapolated distance $\hat{d} = s(n+m)/r$. This extrapolation is reliable when edits are spread evenly along the pair and can be too high by orders of magnitude when they are concentrated, as near the ends of nanopore reads. dinara-align therefore computes two extrapolations, $\hat{d}_1$ from the first eight edits of a diagonal-transition search and $\hat{d}_2$ from the same search continued until its budget, and sets the first bound from them only if $\max(\hat{d}_1, \hat{d}_2) \le \frac{4}{3}\min(\hat{d}_1, \hat{d}_2)$. Otherwise, as in A*PA2, the bound starts 256 above the known lower bound, and each retry at most doubles it. A failed round sets the next bound from where it stopped, its progress extrapolated to the whole width, rather than doubling. Within a round, the distance is extrapolated again after one eighth, one quarter and one half of the columns; the bound is lowered if the extrapolation lies below it, and the round is abandoned if the extrapolation exceeds it. Because these rules change only the sequence of bounds, the result remains exact.

Within a round, the band reads the scores down the right edge of each tile and drops the rows at its top and bottom whose score plus the heuristic exceeds the bound, as A*PA2 does. A score changes by at most one per row, and the heuristic by at most its climb c, one for exact seeds and two for inexact ones, so a row that exceeds the bound by x rules out the next $\lceil x/(1+c) \rceil$ rows without reading them. The bottom of the next tile is bounded by the deepest row kept, since going k rows past it costs at least k more. With seeds, when at least one seed in 32 chains from the origin, the first bound starts 64 above the heuristic at the origin, which on long reads lies within a few percent of the distance.

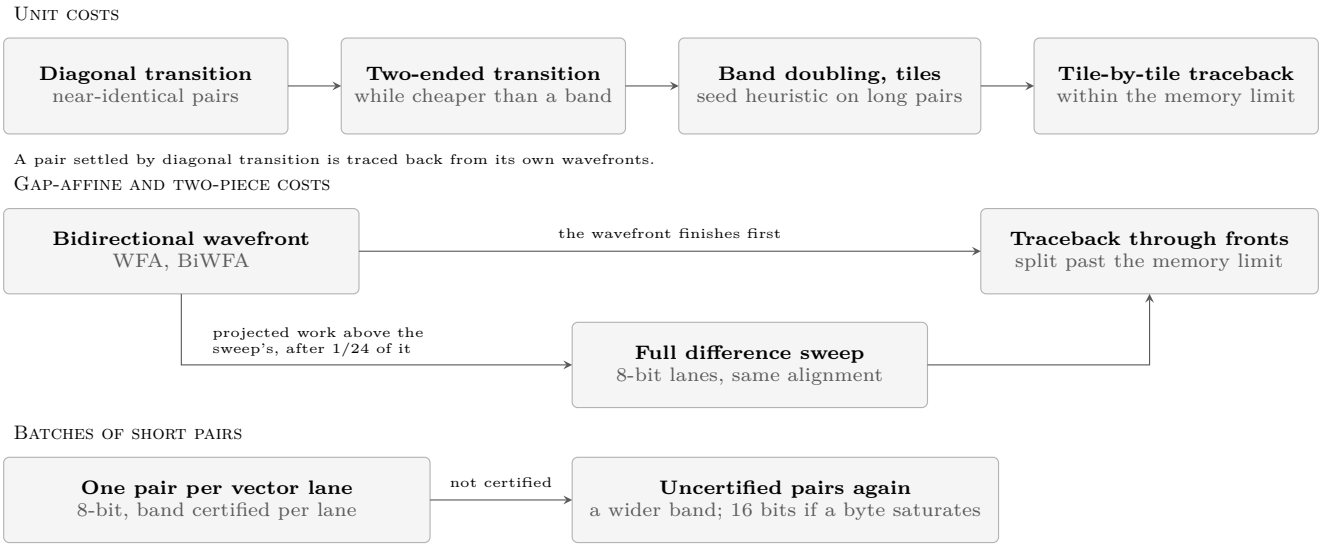


Figure 1: How dinara-align assigns a pair to an algorithm. At unit costs, diagonal transition from one end and then from both settles near-identical pairs while it costs less than a band would; other pairs go to band doubling over bit-parallel tiles, pruned by the seed heuristic on long pairs, and are traced back a tile at a time within the memory limit. At gap-affine costs, the bidirectional wavefront hands a pair to a full sweep in difference recurrences once its projected work exceeds the sweep's, the alignment being the same either way. Batches of short pairs are aligned one pair per vector lane within a band whose optimality each lane certifies.

Table 1: Algorithms in dinara-align and the changes introduced in this work.

Problem	Algorithm	Changes (section)
Unit costs, long pairs	Band doubling over bit-parallel tiles with the gap-chaining seed heuristic (A*PA2)	Regrouped recurrence, staggered groups (3.2); agreeing bounds (3.3); neighbour construction (3.5); traceback within a memory limit (3.7)
Unit costs, similar pairs	Diagonal transition (WFA, BiWFA)	Gathered extension (3.4)
Gap-affine, two-piece	Bidirectional wavefront (WFA, BiWFA); full sweep in difference recurrences for divergent pairs (Suzuki and Kasahara)	Gathered extension (3.4); score-derived band (3.8); handover gated on work spent (S1)
Substitution matrices, local	Anti-diagonal vector sweep; end-first local alignment (SSW)	Backward wavefront from the best end (3.8)
Batches of short pairs	Inter-sequence vectorisation (SWIPE, SeqAn, parasail)	Certified per-lane bands (3.6)

3.4 Diagonal transition for near-identical pairs

Diagonal transition extends each diagonal of a wavefront along its run of matches. dinara-align compares eight letters at a time by the exclusive-or of two 64-bit words and a count of trailing zeros. On AVX-512, eight diagonals are extended together: their letters are loaded with gather instructions, and the wavefront is padded with unreachable diagonals so that every step processes full vectors under branch-free masks. The gather masks are derived from the lanes of each group, which avoids a dependency between consecutive gathers that would otherwise serialise them. The same vectorised step serves near-identical pairs, computed from one or both ends (Marco-Sola et al., 2023), the traceback inside each tile, restricted to the diagonals that can still reach the traced cell, and the three-layer gap-affine wavefront.

A pair whose diagonal transition finishes within its budget is aligned from its wavefronts alone. From one end the wavefronts are kept whole. From both ends, as in BiWFA, the forward wavefronts are grown on to the proven distance, each cost kept only on the diagonals on which the backward wavefronts show that an optimal path passes, so that the extra costs grow few diagonals. The backtrace then takes, at each cost, the furthest-reaching source, which is the order of Section 3.1, and the CIGAR is written run by run from the wavefronts.

3.5 Seed heuristic: direct neighbour construction

The reference is divided into disjoint seeds of 12 bases, matched exactly, or of 16 bases, matched with at most one edit (Groot Koerkamp and Ivanov, 2024). A 16-base seed that matches with at most one edit matches one of its two 8-base halves exactly, so each half indexes a table. Since these tables exceed the caches, the lookups for 16 query rows are issued together, so that their memory accesses overlap, and only seeds whose matches could still chain to the end of the pair are considered.

An exact match μ of a seed at query rows $[i, i+16)$ always has four neighbouring matches with one edit: two share its start, at rows $[i, i+15)$ and $[i, i+17)$, and two share its end, at $[i+1, i+16)$ and $[i-1, i+16)$.

Because chaining is all-or-nothing, a neighbour one diagonal away may chain where μ cannot, and the neighbours are required for the heuristic to remain a lower bound. Rather than locating them by search and testing each for local pruning, dinara-align inserts them whenever μ is retained, which is justified by the following lemma. Local pruning retains a match ν only if $c(\nu) + h(e(\nu)) < \tau$ (Section 2.4).

Lemma 1. If an exact match μ fails local pruning, so does each of its four neighbours.

Proof. A neighbour ν that shares the end of μ has $e(\nu) = e(\mu)$ and $c(\nu) = 1 > c(\mu) = 0$. A neighbour that shares the start ends on an adjacent diagonal, and one insertion or deletion from $e(\mu)$ reaches a cell on that diagonal at or beyond $e(\nu)$. The least remaining cost does not increase along a diagonal (Ukkonen, 1985), so $h(e(\mu)) \le 1+h(e(\nu))$ and $c(\mu)+h(e(\mu)) \le c(\nu)+h(e(\nu))$. In both cases the pruning test value of ν is at least that of μ. □

Neighbours sharing the start therefore raise the layer of that start by the chain length available from their ends, and neighbours sharing the end are placed one layer below it, without a search or a pruning test of their own.

3.6 Certified bands for inter-sequence batches

Batches of short pairs are aligned with one pair per vector lane (Rognes, 2011; Rahn et al., 2018), first in saturating 8-bit lanes and, for the scores of pairs that saturate, in 16-bit lanes (Zhao et al., 2013; Daily, 2016); alignments stay in 8-bit lanes. Rather than computing each matrix in full, a lane computes the band of diagonals $[d_\ell, d_h]$ containing the main diagonal 0 and the end diagonal $\delta = m - n$, together with one further diagonal on either side. Its optimality is certified with the following bound, a cost form of the out-of-band bounds of Gibrat (2018) and Fujiki et al. (2020), stated for gap-affine and two-piece costs and with a strict form for tie-breaking.

Proposition 1. Let C be the least cost of an alignment within $[d_\ell, d_h]$, and let $L(d) = g(|d|) + g(|d - \delta|)$. If $C \le \min\big(L(d_h + 1), L(d_\ell - 1)\big)$, then C is the optimal cost; if the inequality is strict, every optimal alignment lies within the band.

Proof. An alignment that leaves the band visits diagonal $d_h + 1$ or $d_\ell - 1$. Consider $d = d_h + 1 > \max(0, \delta)$; the case below the band is symmetric. Starting on diagonal 0, the alignment contains insertions totalling at least d letters to reach d, and deletions totalling at least $d - \delta$ letters to end on δ. These form at least one gap of each kind, and since g is subadditive (for two-piece costs, with the usual convention that the longer piece has the lower extension cost), their total cost is at least $g(d) + g(d - \delta) = L(d)$. Every alignment outside the band therefore costs at least $\min\big(L(d_h + 1), L(d_\ell - 1)\big)$. □

Bounds of this kind underlie band doubling (Ukkonen, 1985) and have been used to verify narrow-band results in software (Gibrat, 2018) and in hardware (Fujiki et al., 2020); here the certificate is evaluated independently in each vector lane of an inter-sequence batch. For alignments, the strict form guarantees that the alignment selected by the tie rule lies within the band. Pairs that fail the certificate, about one in seven in our short-read benchmark, are recomputed over every diagonal that a cheaper path could visit, grouped by the width of that band so that the lanes of a group finish together.

3.7 Traceback within a memory limit

The traceback of a band runs from its last tile back, and traces each tile from the boundary recorded on its left: the vertical differences down that boundary, within the band. On a 1 Mbp pair at 20% errors these boundaries take up to 175 MB, so the memory limit of an alignment, 80 MB by default, applies to them. While they fit, every boundary is kept; beyond the limit only every k-th is, k doubling as the round proceeds, with room left for k of the widest boundaries. Each tile also records the state that the band carries into it: its first and last words, the score at its top, the deepest row kept and its score, and the bound. The traceback computes a stretch of tiles again from the last kept boundary before a missing one. Every step of a band depends only on this state and on the scores it computes, so each boundary is recovered as the round left it, and the alignment is the same byte for byte. Splitting the pair instead, as QuickEd does (Doblas et al., 2025) and as our gap-affine wavefront does past its own limit, aligns the parts independently and resolves ties within each. On the 1 Mbp pairs at 20% errors, the growth of peak memory fell from 175 to 133 MB on the worst pair, and the four pairs took 9% longer; pairs whose boundaries fit are unchanged.

3.8 Gap-affine and score-based alignment

A scoring scheme with match score a, mismatch score b and gap scores $-(o + ke)$ can be converted into costs for global alignment (Eizenga and Paten, 2022). Every alignment covers $n + m$ letters, so its score S and a gap-affine cost C satisfy

$$\begin{aligned} S &= \tfrac{1}{2}\big(a(n+m) - C\big), \\ C &= 2(a-b)\,X + \sum_{\text{gaps}} \big(2o + k(2e+a)\big), \end{aligned} \tag{4}$$

where X is the number of mismatches, so the wavefront computes score-maximising alignments. The cost also bounds the band in which the alignment must be stored. Each gapped letter contributes at least $2e + a$ to C, and an alignment that deviates by t diagonals from both the main and the end diagonal contains at least $2t$ gapped letters; hence only diagonals within $C/(2(2e + a))$ of them can contain an optimal alignment. For a 1 kbp pair at 5% divergence, this stores 137 diagonals instead of 1,000. Local alignment first locates the end of the optimal alignment with a forward sweep, as in SSW (Zhao et al., 2013), and then extends a wavefront backwards from that end, stopping at the first cell that attains the maximal score; this avoids

the correction pass that striped layouts may need when long alignments score highly (Farrar, 2007).

The bidirectional wavefront keeps, for every cost and diagonal, the furthest column and one byte naming the source of each layer, five bytes per diagonal, and traces the alignment through these fronts from where the two searches met; a pair whose fronts would exceed the memory limit is split where an optimal path crosses, as in BiWFA. A pair that ends within the costs the forward search can hold is finished by that search alone, before the backward one starts, which saved a third of the time of short reads at 1% errors, and a cost without traceback stops as soon as its best lies within one of the costs both searches have checked against the start of the other. A global pair whose projected work exceeds that of a full sweep in difference recurrences (Suzuki and Kasahara, 2018), in 8-bit lanes, is handed to that sweep once the wavefront has spent one twenty-fourth of it; the sweep stores the predecessor flags of Section 3.1, so the alignment is the same, and unrelated pairs of 5 kbp aligned in 4.8 ms instead of 32.9 ms.

Under a general substitution matrix, anti-diagonals are computed in 16-bit lanes while the scores provably fit (Wozniak, 1997), a strip of rows of at most 16 KB at a time, so that the rows a strip reads stay in the first-level cache; this took local alignment of 10 kbp against 12 kbp from 50.0 to 17.4 ms. A global alignment is traced in a band of diagonals that grows, at most fourfold per step, until the score it yields proves that every optimal path lies inside it, so a pair close to its diagonal takes one sweep of about its own cells rather than a sweep of the matrix for its score: 1 kbp pairs at 10% divergence aligned in about 200 µs instead of 544 µs. A band whose decisions would exceed the memory limit keeps every k-th anti-diagonal and computes each stretch again for the traceback.

3.9 Thresholds measured against the code

The choice of algorithm and the bounds of each search rest on 41 thresholds, from the step budgets of the diagonal transition to the seed gates and the limits of the traceback, each set from measurements when it was introduced, on the code of the time. We measured all of them again on the final code: dinara-align was built with each threshold halved and with each doubled, and the 83 builds were timed on 17 unit-cost and 8 gap-affine samples against the unchanged build, which was timed again after every six. About thirty thresholds moved no sample by more than 3% in either direction, and most of the others traded one dataset against another. One was stale: the length from which seeds match with one edit, which had cost 100 kbp pairs at 10% errors a third of their running time. Two further stale choices were found by examining the slowest results directly: a narrower tile for narrow bands, kept from the first version, which had cost 10 kbp pairs at 5% errors a fifth of their time, and a projection that handed nanopore reads to a full sweep at gap-affine costs, which had slowed them by half (Supplementary Section S1).

3.10 Validation

A differential fuzzer compares every mode, cost model, band, cost limit and tie rule on random pairs with a full-matrix implementation that shares no code with the library, and the WFA2-lib regression set is reproduced exactly, every CIGAR included (Supplementary Section S1).

4 Results

4.1 Experimental setup

All measurements were taken on an Intel Core i9-7900X (Skylake-X, AVX-512), its ten cores fixed at 3.3 GHz with turbo boost and hyper-threading disabled, with dinara-align compiled by Mojo 1.1.0 (Modular Inc., 2026) at commit `dc980d7` (gap-affine costs and batches at `289bcf0`, whose code those workloads share) and every tool compiled for the native instruction set. We combined the protocols of A*PA2 (Groot Koerkamp, 2024) and QuickEd (Doblas et al., 2025): the real datasets of A*PA2, nanopore reads of about 1, 10, 50 and 500 kbp, the last also with genetic variation, and SARS-CoV-2 genomes, and the simulated datasets of QuickEd, random sequences of 10 kbp to 1 Mbp with 1, 5, 10 and 20% errors, each sampled to about 2 Mbp and at least four pairs (Supplementary Table S5). Each tool aligned each pair once, with traceback, on one pinned core, within 30 s per sample. The exact aligners compared are A*PA2-full and A*PA2-simple, A*PA (Groot Koerkamp and Ivanov, 2024), Edlib (Šošić and Šikić, 2017), WFA and BiWFA (Marco-Sola et al., 2021, 2023), QuickEd, BSAlign (Shao and Ruan, 2024), KSW2 (Li, 2018), parasail (Daily, 2016) and SSW (Zhao et al., 2013), each on the workloads it supports; every cost agreed with those of every other tool and with the published A*PA2 results.

4.2 Speed and memory

dinara-align had the lowest mean time per alignment of the exact aligners on every workload (Table 2; Table 3). On the nanopore reads it was 1.3 to 2.6 times faster than the fastest alternative, and on the simulated pairs 1.1 to 2.3 times (Fig. 2): 100 kbp pairs at 1, 5, 10 and 20% errors took 1.5, 5.0, 7.2 and 16.0 ms, against 7.5, 8.5, 13.9 and 36.7 ms for A*PA2-full and 12.5, 24.0, 39.0 and 62.8 ms for QuickEd, and its narrowest margin was on 10 kbp pairs at 5%, 269 against 288 µs for A*PA2-simple. Over all 22 datasets QuickEd took 3.0 to 22.5 times as long, 6.7 times in the geometric mean. At gap-affine costs dinara-align was 1.7 to 3.8 times faster than WFA and BiWFA wherever a tool completed a dataset, and it completed more of the long pairs than any other tool; in the other modes it was 1.0 to 22 times faster than the fastest tool offering the same mode, the one tie lying within the variation between builds. Global scores of 500,000 short-read pairs took 230 ms on one core and 27 ms on ten at (1, 2, 1), 4.6 and 4.3 times faster than WFA2-lib.

Table 2: Unit-cost alignment of the A*PA2 real datasets by every exact aligner: mean time per alignment, traceback included, and below it the peak resident memory of the run in MB, as QuickEd reports it. t.o. marks a tool that completed no pair within 30 s and o.o.m. one the 8 GB limit stopped. The simulated datasets are in Fig. 2 and Supplementary Table S4.

Dataset	dinara-align	A*PA2-full	A*PA2-simple	A*PA	QuickEd	Edlib	BiWFA	WFA	BSAlign	KSW2
Mean time per alignment										
ont-1k (0.8 kbp)	**23 µs**	81 µs	48 µs	612 µs	75 µs	124 µs	44 µs	31 µs	119 µs	539 µs
ont-10k (3.6 kbp)	**157 µs**	391 µs	247 µs	15.3 ms	688 µs	1.09 ms	741 µs	599 µs	1.30 ms	13.1 ms
ont-50k (9.6 kbp)	**590 µs**	1.27 ms	944 µs	263 ms	2.31 ms	6.14 ms	7.43 ms	6.38 ms	12.3 ms	167 ms
ont-500k (642 kbp)	**118 ms**	283 ms	687 ms	t.o.	847 ms	5.00 s	6.33 s	o.o.m.	t.o.	o.o.m.
ont-500k-genvar (663 kbp)	**156 ms**	225 ms	511 ms	t.o.	1.39 s	5.23 s	6.20 s	o.o.m.	t.o.	o.o.m.
SARS-CoV-2 (30 kbp)	**285 µs**	2.00 ms	728 µs	6.49 ms	3.90 ms	8.18 ms	909 µs	987 µs	47.1 ms	68.5 ms
Peak resident memory (MB)										
ont-1k	14	5	5	6	2	7	7	8	2	7
ont-10k	16	6	6	39	6	8	8	39	25	72
ont-50k	20	8	7	858	18	8	10	187	423	650
ont-500k	67	89	90	t.o.	26	15	29	o.o.m.	t.o.	o.o.m.
ont-500k-genvar	63	57	55	t.o.	27	16	27	o.o.m.	t.o.	o.o.m.
SARS-CoV-2	18	7	6	19	15	8	9	26	215	151

Table 3: Gap-affine alignment at costs (4, 6, 2) by every exact aligner that takes them: mean time per alignment, a 30 s budget per sample. Superscripts give the pairs completed where not all were; t.o. marks a tool that completed none and o.o.m. one the 8 GB limit stopped. Every dataset is in Supplementary Table S3.

Dataset	dinara-align	WFA	BiWFA	BSAlign	KSW2
ont-1k	**112 µs**	186 µs	304 µs	336 µs	1.10 ms
ont-10k	**1.71 ms**	4.80 ms	5.84 ms	7.84 ms	29.8 ms
ont-50k	**22.0 ms**	63.5 ms	54.9 ms	87.0 ms	343 ms$^{81/104}$
ont-500k	t.o.	o.o.m.	t.o.	o.o.m.	o.o.m.
ont-500k-genvar	**21.7 s**$^{1/4}$	o.o.m.	t.o.	o.o.m.	o.o.m.
SARS-CoV-2	**737 µs**	2.06 ms	1.79 ms	341 ms	88.4 ms
10 kbp, 1%	**208 µs**	461 µs	650 µs	37.0 ms	21.3 ms
10 kbp, 5%	**2.32 ms**	7.05 ms	8.72 ms	37.0 ms	97.5 ms
10 kbp, 10%	**7.18 ms**	22.7 ms	27.3 ms	37.3 ms	204 ms
10 kbp, 20%	**20.6 ms**	65.2 ms	75.2 ms	37.4 ms	374 ms$^{81/99}$
100 kbp, 1%	**14.0 ms**	39.7 ms	39.0 ms	o.o.m.	2.44 s
100 kbp, 5%	**337 ms**	811 ms	715 ms	o.o.m.	9.25 s$^{3/9}$

An alignment of an ultra-long read increased peak memory by at most 51 MB, against 82 MB for A*PA2-full, and of a 1 Mbp pair at 20% errors by at most 126 MB, against 184 MB, its traceback held within the default limit of 80 MB (Section 3.7; Supplementary Table S6); QuickEd stayed near 25 MB. On short reads dinara-align's peak is higher than the others' (Table 2), its runtime alone taking about 13 MB.

4.3 Effect of each technique

To measure what each technique contributes, dinara-align was built with one switched off at a time and every build aligned samples of the A*PA2 real datasets and of the 100 kbp simulated pairs, and scored the short-read batch, the builds alternating within three rounds on one pinned core, assembled with branches kept clear of 32-byte boundaries, which the Skylake-X penalises (Table 4); every build returned the same costs. Constructing seed neighbours directly contributed most on the ultra-long reads, 9% and 16%; 100 kbp pairs, which take exact seeds on AVX-512, do not use it. Interleaving two groups of bit-parallel words contributed 4.5 to 5% on the ultra-long reads and 11.5% on 100 kbp pairs at 20% errors, whose bands are widest; the regrouped recurrence contributed 1 to 4%, as bands narrow enough to be bound by a single word's step are a small part of the work. Gathered diagonal transition contributed 13 to 27% where diagonal transition does most of the work, and the agreement rule 22% on the 10 kbp reads. Without the certified band, the short-read batch took 2.9 and 3.2 times as long.

5 Discussion

The algorithms in dinara-align build on those of earlier exact aligners; the gains come from changes to how these algorithms use the processor. Two observations stand out. First, on long pairs the techniques that reduce work outside the inner loop contributed more than those that speed it up: constructing seed neighbours directly saved up to 16%, against 4.5 to 11.5% for interleaving bit-parallel groups and 1 to 4% for the regrouped recurrence (Table 4). Second, thresholds that encode a cost model go stale as the code around them changes, and measuring them all again belongs in the evaluation of any aligner that chooses its algorithm by fixed thresholds (Section 3.9). Neither the regrouped recurrence nor the per-lane band certificate depends on the implementation, and both apply directly to other bit-parallel and inter-sequence aligners.

The study has several limitations:

1. The evaluation covers a single x86 processor with AVX-512; the AVX2 configuration was measured

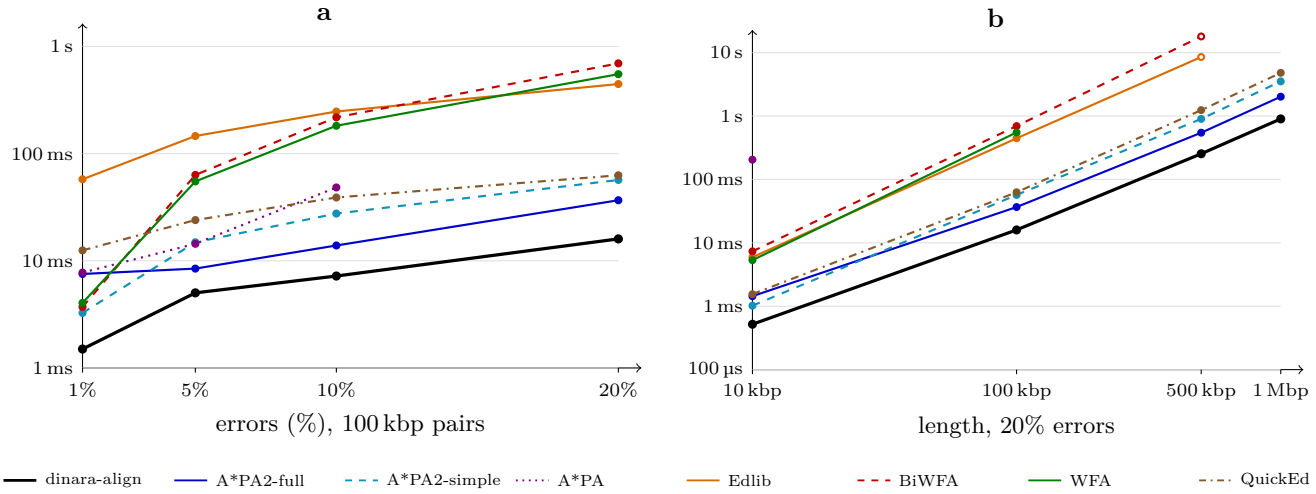


Figure 2: Unit-cost alignment of QuickEd's simulated pairs, mean time per alignment on a logarithmic scale. (a) 100 kbp pairs by error rate. (b) Pairs at 20% errors by length (5% in the interactive version); lines end where a tool completed no pair within 30 s, and hollow points mark tools that completed only some pairs. dinara-align had the lowest mean time at every length and error rate.

Table 4: Effect of each technique: median time with all techniques on (ms per alignment; per batch of 500,000 pairs for the short reads), and the change when one technique is switched off, over three alternating rounds on the Skylake-X at commit `dc980d7`, with branches kept clear of 32-byte boundaries. Every configuration returned the same costs.

Dataset	All on	Regrouped recurrence	Paired groups	Gathers	Agreement rule	Seed neighbours	Certified band
ont-500k	98.8	+1.1%	**+4.5%**	+0.5%	−0.2%	**+8.9%**	
ont-500k-genvar	136.0	+1.9%	**+5.0%**	+0.4%	0.0%	**+16.4%**	
ont-1k	0.021	+0.2%	+0.3%	**+14.9%**	0.0%	+0.7%	
ont-10k	0.145	−0.6%	+1.2%	**+6.4%**	**+22.2%**	+0.1%	
SARS-CoV-2	0.197	−0.9%	+0.2%	**+13.4%**	**+4.9%**	−0.5%	
100 kbp, 1%	1.10	−1.4%	+0.4%	**+26.5%**	−0.1%	−1.0%	
100 kbp, 5%	4.61	−0.6%	−0.2%	+0.1%	0.0%	+0.2%	
100 kbp, 10%	6.72	+0.3%	+3.4%	+1.9%	+0.5%	−0.2%	
100 kbp, 20%	15.0	**+4.1%**	**+11.5%**	+3.9%	−2.9%	0.0%	
Short reads, affine	232						**+188%**
Short reads, unit	213						**+215%**

only for its inexact-seed threshold, and the NEON configuration is not evaluated here.

2. Some thresholds were chosen using workloads that also appear in the evaluation; in particular, the AVX-512 thresholds for inexact seeds, the share of chained seeds (lowered from 40 to 20%) and the length from which they are used (raised from 64 to 128 kbp), were set after measurements on uniform 100 kbp pairs like those of Fig. 2a, so results on those pairs may overstate performance on unseen data.

3. As in the A*PA2 protocol, each pair was aligned once, and on the Skylake-X some running times varied by up to one third between builds that did not change the code involved, which we attribute to code alignment effects in the instruction decoder; the ablation controls for this, the comparison with other tools does not.

4. The ablation switches off one technique at a time against the full configuration, so it does not separate interactions between techniques.

5. Release builds target the baseline processor of each platform, so the AVX-512 code paths require a build for the host processor.

Future work includes:

1. faster construction of the seed heuristic, whose scalar lookups dominate on long divergent pairs;

2. reusing computation across band-doubling rounds, as in the incremental doubling of A*PA2;

3. a seed heuristic for gap-affine costs, at which no tool completed a 1 Mbp pair beyond 1% errors;

4. measurements on further processors, and an evaluation of the GPU kernels the library provides for batch scores and alignments (Supplementary Section S1).

Funding

This work received no specific funding.

Conflict of interest

None declared.

Data availability

The software, benchmark harness and the commits of every compared tool are available at `https://github.com/kokyriakidis/dinara-align`. The benchmarks are reproduced by `pixi run bench` (unit and `-affine` costs), `bench-modes`, `bench-batch` and `bench-ablation`.

References

Jeff Daily. Parasail: SIMD C library for global, semi-global, and local pairwise sequence alignments. *BMC Bioinformatics*, 17:81, 2016. doi: 10.1186/s12859-016-0930-z.

Max Doblas, Oscar Lostes-Cazorla, Quim Aguado-Puig, Nick Cebry, Pau Fontova-Musté, Christopher Batten, Santiago Marco-Sola, and Miquel Moretó. QuickEd: high-performance exact sequence alignment based on bound-and-align. *Bioinformatics*, 41(3):btaf112, 2025. doi: 10.1093/bioinformatics/btaf112.

Jordan M. Eizenga and Benedict Paten. Improving the time and space complexity of the WFA algorithm and generalizing its scoring. bioRxiv, 2022.

Michael Farrar. Striped Smith–Waterman speeds database searches six times over other SIMD implementations. *Bioinformatics*, 23(2):156–161, 2007. doi: 10.1093/bioinformatics/btl582.

Daichi Fujiki, Shunhao Wu, Nathan Ozog, Kush Goliya, David Blaauw, Satish Narayanasamy, and Reetuparna Das. SeedEx: A genome sequencing accelerator for optimal alignments in subminimal space. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 937–950, 2020. doi: 10.1109/MICRO50266.2020.00080.

Jean-François Gibrat. A short note on dynamic programming in a band. *BMC Bioinformatics*, 19:226, 2018. doi: 10.1186/s12859-018-2228-9.

Osamu Gotoh. An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3):705–708, 1982. doi: 10.1016/0022-2836(82)90398-9.

Ragnar Groot Koerkamp. A*PA2: Up to 19× faster exact global alignment. In Solon P. Pissis and Wing-Kin Sung, editors, *24th International Workshop on Algorithms in Bioinformatics (WABI 2024)*, volume 312 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 17:1–17:25. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. doi: 10.4230/LIPIcs.WABI.2024.17.

Ragnar Groot Koerkamp and Pesho Ivanov. Exact global alignment using A* with chaining seed heuristic and match pruning. *Bioinformatics*, 40(3):btae032, 2024. doi: 10.1093/bioinformatics/btae032.

Heng Li. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18):3094–3100, 2018. doi: 10.1093/bioinformatics/bty191.

Daniel Liu and Martin Steinegger. Block aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices. *Bioinformatics*, 39(8):btad487, 2023. doi: 10.1093/bioinformatics/btad487.

Santiago Marco-Sola, Juan Carlos Moure, Miquel Moreto, and Antonio Espinosa. Fast gap-affine pairwise alignment using the wavefront algorithm. *Bioinformatics*, 37(4):456–463, 2021. doi: 10.1093/bioinformatics/btaa777.

Santiago Marco-Sola, Jordan M. Eizenga, Andrea Guarracino, Benedict Paten, Erik Garrison, and Miquel Moreto. Optimal gap-affine alignment in $O(s)$ space. *Bioinformatics*, 39(2):btad074, 2023. doi: 10.1093/bioinformatics/btad074.

Modular Inc. Mojo programming language. `https://docs.modular.com/mojo/`, 2026.

Eugene W. Myers. An $O(ND)$ difference algorithm and its variations. *Algorithmica*, 1(1–4):251–266, 1986. doi: 10.1007/BF01840446.

Gene Myers. A fast bit-vector algorithm for approximate string matching based on dynamic programming. *Journal of the ACM*, 46(3):395–415, 1999. doi: 10.1145/316542.316550.

René Rahn, Stefan Budach, Pascal Costanza, Marcel Ehrhardt, Jonny Hancox, and Knut Reinert. Generic accelerated sequence alignment in SeqAn using vectorization and multi-threading. *Bioinformatics*, 34(20):3437–3445, 2018. doi: 10.1093/bioinformatics/bty380.

Torbjørn Rognes. Faster Smith-Waterman database searches with inter-sequence SIMD parallelisation. *BMC Bioinformatics*, 12:221, 2011. doi: 10.1186/1471-2105-12-221.

Haojing Shao and Jue Ruan. BSAlign: a library for nucleotide sequence alignment. *Genomics, Proteomics & Bioinformatics*, 22(2):qzae025, 2024. doi: 10.1093/gpbjnl/qzae025.

Martin Šošić and Mile Šikić. Edlib: a C/C++ library for fast, exact sequence alignment using edit distance. *Bioinformatics*, 33(9):1394–1395, 2017. doi: 10.1093/bioinformatics/btw753.

Hajime Suzuki and Masahiro Kasahara. Introducing difference recurrence relations for faster semi-global alignment of long sequences. *BMC Bioinformatics*, 19(Suppl 1):45, 2018. doi: 10.1186/s12859-018-2014-8.

Esko Ukkonen. Algorithms for approximate string matching. *Information and Control*, 64(1–3):100–118, 1985. doi: 10.1016/S0019-9958(85)80046-2.

Andrzej Wozniak. Using video-oriented instructions to speed up sequence comparison. *Computer Applications in the Biosciences (CABIOS)*, 13(2):145–150, 1997. doi: 10.1093/bioinformatics/13.2.145.

Mengyao Zhao, Wan-Ping Lee, Erik P. Garrison, and Gabor T. Marth. SSW library: An SIMD Smith-Waterman C/C++ library for use in genomic applications. *PLoS ONE*, 8(12):e82138, 2013. doi: 10.1371/journal.pone.0082138.